

Algorithm for File Updates in Python

Automating allow list maintenance for a restricted patient records subnetwork

Project description

As a security professional at a health care company, one of my regular responsibilities is maintaining the file that controls which employees can access restricted patient records. Access to the restricted subnetwork is granted by IP address, and the file "allow_list.txt" holds the addresses that are permitted to sign in. When an employee should no longer have that access, their IP address is added to a separate remove list and has to be taken out of the allow list. In this project I developed a Python algorithm that opens "allow_list.txt", checks its contents against the remove list, deletes any IP address that appears on both, and writes the revised list back to the file. Automating the check makes a task that has to be repeated often faster and far less prone to human error.

Open the file that contains the allow list

The first step of the algorithm is to open the file so that its contents can be accessed. I assigned the file name to a variable and then opened it with a with statement:

```
import_file = "allow_list.txt"
```

```
with open(import_file, "r") as file:
```

Storing the file name in the `import_file` variable means the file only has to be identified once and can then be referred to by that variable throughout the algorithm. The `with` keyword manages the resources used while the file is open and automatically closes it once the indented block has finished running, so there is no need to call `.close()` separately. Inside the statement, the `open()` function takes two arguments: the file to be opened and the mode it should be opened in. The `"r"` argument opens the file in read mode. Finally, the `as` keyword assigns the file object returned by `open()` to the variable `file`, which the next step uses to access the contents.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Read the initial contents of the file
with open(import_file, "r") as file:
```

Screenshot: opening allow_list.txt in read mode with a with statement.

Read the file contents

With the file open, the next step is to convert its contents into something the algorithm can work with. :

```
ip_addresses = file.read()
```

The `.read()` method is called on the file object and converts the contents of the file into a string, which I stored in the variable `ip_addresses`. This line is indented because it belongs to the body of the `with` statement; the file has to be read while it is still open. At this point `ip_addresses` holds every IP address in the allow list as a single string, with the addresses separated by whitespace.

```
# Read the imported file and store it in a variable named ip_addresses  
ip_addresses = file.read()
```

Screenshot: reading the file contents into the ip_addresses variable with .read().

Convert the string into a list

Individual IP addresses cannot be removed from a string, so the data has to be restructured before any of it can be deleted:

```
ip_addresses = ip_addresses.split()
```

The `.split()` method converts a string into a list. Because no argument is passed to it, it splits the string on whitespace by default, which includes the line breaks that separate the IP addresses in the file. The result is reassigned to `ip_addresses`, so the same variable now stores a list in which each IP address is its own element and can be accessed or removed individually. This line sits outside the `with` statement because the data is already in memory and the file itself is no longer needed.

```
# Convert `ip_addresses` from a string to a list  
ip_addresses = ip_addresses.split()
```

Screenshot: converting the ip_addresses string into a list with .split().

Iterate through the remove list

A second list called `remove_list` contains the IP addresses that should no longer have access to the restricted content. To compare it against the allow list, I set up an iterative statement:

```
remove_list = ["192.168.97.225", "192.168.158.170",  
               "192.168.201.40", "192.168.58.57"]
```

```
for element in remove_list:
```

The `for` keyword starts a loop that runs the indented code once for every item in the sequence that follows the `in` keyword. `element` is the loop variable, and on each pass through the loop it holds the current IP address from `remove_list`. Iterating through the remove list rather than the allow list means

the loop only runs as many times as there are addresses to remove, which keeps the algorithm efficient as the allow list grows.

```
# List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# Iterative statement to loop through `remove_list`
for element in remove_list:
```

Screenshot: the for loop header that iterates through `remove_list`.

Remove IP addresses that are on the remove list

The body of the loop compares each address on the remove list against the allow list and deletes it where there is a match:

```
for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)
```

The `if` statement evaluates whether the current value of `element` is present in the `ip_addresses` list. In this context `in` acts as a membership operator and returns `True` when the value is found. When the condition evaluates to `True`, the `.remove()` method is applied to `ip_addresses` and deletes the element passed to it. Evaluating the conditional first also prevents the algorithm from stopping with an error, because calling `.remove()` with a value that is not in the list raises a `ValueError`.

Applying the `.remove()` method in this way is possible because there are no duplicates in the `ip_addresses` list. The method only removes the first occurrence of the value it is given, so if the allow list contained the same IP address more than once, a single pass of the loop would leave the extra copies in place and those employees would keep their access.

```
for element in remove_list:
    # If current element is in `ip_addresses`,
    if element in ip_addresses:
        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)
```

Screenshot: the conditional and the `.remove()` method inside the loop.

Update the file with the revised list of IP addresses

The revised data is held in a Python list, but the file has to store it as text, so the list is converted back into a string before it is written:

```
ip_addresses = "\n".join(ip_addresses)
```

```
with open(import_file, "w") as file:  
    file.write(ip_addresses)
```

The `.join()` method is called on the string `"\n"` and combines every element of `ip_addresses` into one string, inserting that string between each element. `\n` is the newline character, so each IP address is placed on its own line and the file keeps the same format it had originally.

A second `with` statement then reopens the file assigned to `import_file`, this time passing `"w"` to the `open()` function so that the file is opened in write mode, and the `.write()` method writes the string to it. Write mode overwrites the existing contents rather than adding to them, which is exactly what is needed here: the file should end up containing only the IP addresses that still have authorization.

```
# Convert `ip_addresses` back to a string so that it can be written into the text file  
ip_addresses = "\n".join(ip_addresses)  
# Rewrite the original file  
with open(import_file, "w") as file:  
    # Rewrite the file, replacing its contents with `ip_addresses`  
    file.write(ip_addresses)
```

Screenshot: rebuilding the string with `.join()` and overwriting the file with `.write()`.

Summary

I developed a Python algorithm that updates the allow list controlling access to a restricted subnetwork. The algorithm opens `"allow_list.txt"` with a `with` statement and the `open()` function, reads its contents into a string with `.read()`, and converts that string into a list with `.split()`. A `for` loop then iterates through the remove list, and a conditional inside the loop applies the `.remove()` method to delete any IP address that appears on both lists. Finally, `.join()` converts the revised list back into a single string separated by newline characters, and a second `with` statement uses `.write()` in write mode to overwrite the file with the updated allow list. The result is a repeatable process that withdraws an employee's access as soon as their IP address is added to the remove list. Because the algorithm can be run again whenever the remove list changes, it makes an authorization task that is performed regularly both faster and more consistent than editing the file by hand.

Complete algorithm

The full algorithm is shown below. Run end to end, it takes the allow list as it stands, strips out every IP address that appears on the remove list, and writes the shortened list back to "allow_list.txt".

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
|
# Read the initial contents of the file
with open(import_file, "r") as file:
    # Read the imported file and store it in a variable named ip_addresses
    ip_addresses = file.read()

# Convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Iterative statement to loop through `remove_list`
for element in remove_list:
    # If current element is in `ip_addresses`,
    if element in ip_addresses:
        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = "\n".join(ip_addresses)

# Rewrite the original file
with open(import_file, "w") as file:
    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)
```

Screenshot: the complete algorithm in the Jupyter Notebook.